S09 · Consistencia distribuida en procesos de negocio

Jannys Graciela Navarro Acrota

Equipo de referencia: 09 · Casa o Nada. Tarea personal sobre Pagatu.

Aporte documentado: evolución de órdenes, cliente y pago, seguridad, comunicación, verificación y documentación individual con asistencia de herramientas.

Ejecución real: 5 de noviembre de 2026 · America/Lima. Fechas ISO en UTC pueden figurar como 6 de noviembre.

Repositorio GitHub · Stephmarc/Tareas-Distribuidas

Evidencia técnica · cuatro bloques

1. Conflicto real y protección idempotente

Se reprodujo de forma aislada una violación SQL de pagos_orden_id_key mediante INSERT SELECT de un pago de prueba. Después se reenviaron los mensajes originales con kafka-console-producer: quedó una sola fila por orden y los logs indicaron duplicado ignorado. Esta reproducción no se presenta como una ejecución histórica de una versión insegura.

2. Fallo y compensación

La orden 55 terminó PAGADA. La orden 56, con método de prueba RECHAZADO, produjo pago.fallido y terminó CANCELADA. Es un rechazo funcional determinista del simulador local, sin cobros bancarios reales.

3. Compensación idempotente

Se reenvió el mismo pago.fallido. El UPDATE de órdenes exige estado PENDIENTE_PAGO: el segundo evento no vuelve a modificar una orden terminal ni genera un segundo aviso. Se confirmó CANCELADA después del duplicado.

4. Proceso adicional y estados

El proceso de avisos también usa UNIQUE(orden_id) y ON CONFLICT DO NOTHING. Se reenvió orden.cancelada y permaneció una sola fila. El diagrama incluye CARRITO, PENDIENTE_PAGO, PAGADA y CANCELADA. Este proceso no reserva stock, cupos ni saldo; no se presenta el bono opcional de stock.

Estados y compensación

```
Inicio -> CARRITO [catálogo no confirmó precio]
Inicio -> PENDIENTE_PAGO [precio confirmado, orden.creada]
PENDIENTE_PAGO -> PAGADA [pago.validado]
PENDIENTE_PAGO -> CANCELADA [pago.fallido]
PAGADA/CANCELADA + evento repetido -> sin cambio
CARRITO: conservado sin cobro; no existe reanudación automática.
```

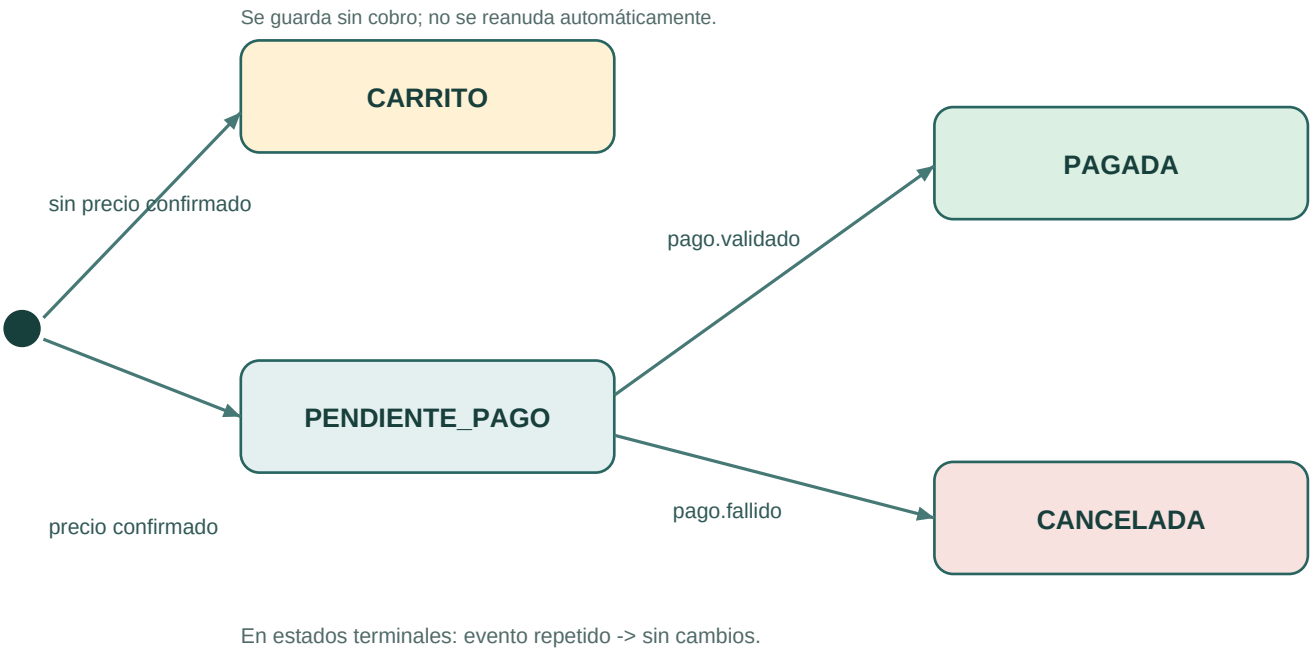


Diagrama de diseño derivado del código; las pruebas de cada transición se presentan a continuación.

Código real del repositorio

OrdenListener.java

Services/pagatu-pago-ms/src/main/java/pe/edu/upeu/pago/event/OrdenListener.java

```
        )
        .setParameter("id", e.ordenId())
        .setParameter("cliente", e.clienteId())
        .setParameter("total", e.total())
        .setParameter("estado", estado)
        .executeUpdate();
      if (n == 0) {
        log.info("Pago duplicado ignorado orden={}", e.ordenId());
        return;
      }
      outbox.registrar(
        "pago-eventos",
        new Evento(
          estado.equals("VALIDADO") ? "pago.validado" : "pago.fallido",
          e.ordenId(),
          e.clienteId(),
          e.total(),
          e.metodoPago(),
          e.traceId()
        )
      );
      log.info("Pago orden={} estado={}", e.ordenId(), estado);
    } finally {
      MDC.remove("traceId");
    }
  }
}
```

PagoListener.java

Services/pagatu-orden-ms/src/main/java/pe/edu/upeu/orden/event/PagoListener.java

```
      default -> null;
    };
    if (estado == null) return;
    try {
      if (e.traceId() != null) MDC.put("traceId", e.traceId());
      int n = em
        .createQuery(
          "update Orden o set o.estado=:estado where o.id=:id and o.estado='PENDIENTE_PAGO'"
        )
        .setParameter("estado", estado)
        .setParameter("id", e.ordenId())
        .executeUpdate();
      log.info(
        "Saga orden={} estado={} resultado={}",
        e.ordenId(),
        estado,
        n == 1 ? "aplicado" : "duplicado ignorado"
      );
      if (n == 1) outbox.registrar(
        "cliente-eventos",
        new Evento(
          estado.equals("PAGADA") ? "orden.confirmada" : "orden.cancelada",
          e.ordenId(),
          e.clienteId(),
          e.total(),
          e.metodoPago(),
          e.traceId()
        )
```

V1__pagos.sql

Services/pagatu-pago-ms/src/main/resources/db/migration/V1__pagos.sql

```
  1 CREATE TABLE pagos(id BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,orden_id BIGINT NOT NULL
UNIQUE,cliente_id VARCHAR(100) NOT NULL,total NUMERIC(10,2) NOT NULL,estado VARCHAR(30) NOT NULL);
```

Ejecución y resultados originales

Comando reproducible: & .\scripts\Run-PersonalEvidence.ps1 -Sesion s09

Los transcripts son salida real de PowerShell y los JSON provienen de las respuestas de la aplicación. Se adjuntan completos en MkDocs. No se sustituyen por un texto de aprobación.

POST orden autenticada | {"status": 201, "request": {"metodoPago": "YAPE_PLIN", "detalles": [{"idProducto": 1, "cantidad": 2}]}, "response": {"id": 55, "nombre": "Orden de 891b60fa-0606-48a7-bd4c-7ef9ed5eabdf", "descripcion": null, "clienteId": "891b60fa-0606-48a7-bd4c-7ef9ed5eabdf", "estado": "PENDIENTE_PAGO", "total": 31, "detalles": [{"id": 55, "nombreProducto": "Producto de prueba", "productoId": 1, "cantidad": 2, "precioUnitario": 15.5, "ordenId": 55}]}, "trace": "personal-s09-d0c37f09-9208-42e8-8f92-4afa78ce74d3"}

Estado final de orden 55 | {"estado": "PAGADA", "total": 31}

POST orden autenticada | {"status": 201, "request": {"metodoPago": "RECHAZADO", "detalles": [{"idProducto": 1, "cantidad": 2}]}, "response": {"id": 56, "nombre": "Orden de 891b60fa-0606-48a7-bd4c-7ef9ed5eabdf", "descripcion": null, "clienteId": "891b60fa-0606-48a7-bd4c-7ef9ed5eabdf", "estado": "PENDIENTE_PAGO", "total": 31, "detalles": [{"id": 56, "nombreProducto": "Producto de prueba", "productoId": 1, "cantidad": 2, "precioUnitario": 15.5, "ordenId": 56}]}, "trace": "personal-s09-58efecf3-12de-4d4f-96ad-b150a4af4ccb"}

Estado final de orden 56 | {"estado": "CANCELADA", "total": 31}

Duplicado de orden no duplica pago | {"orden": 55, "filas": 1}

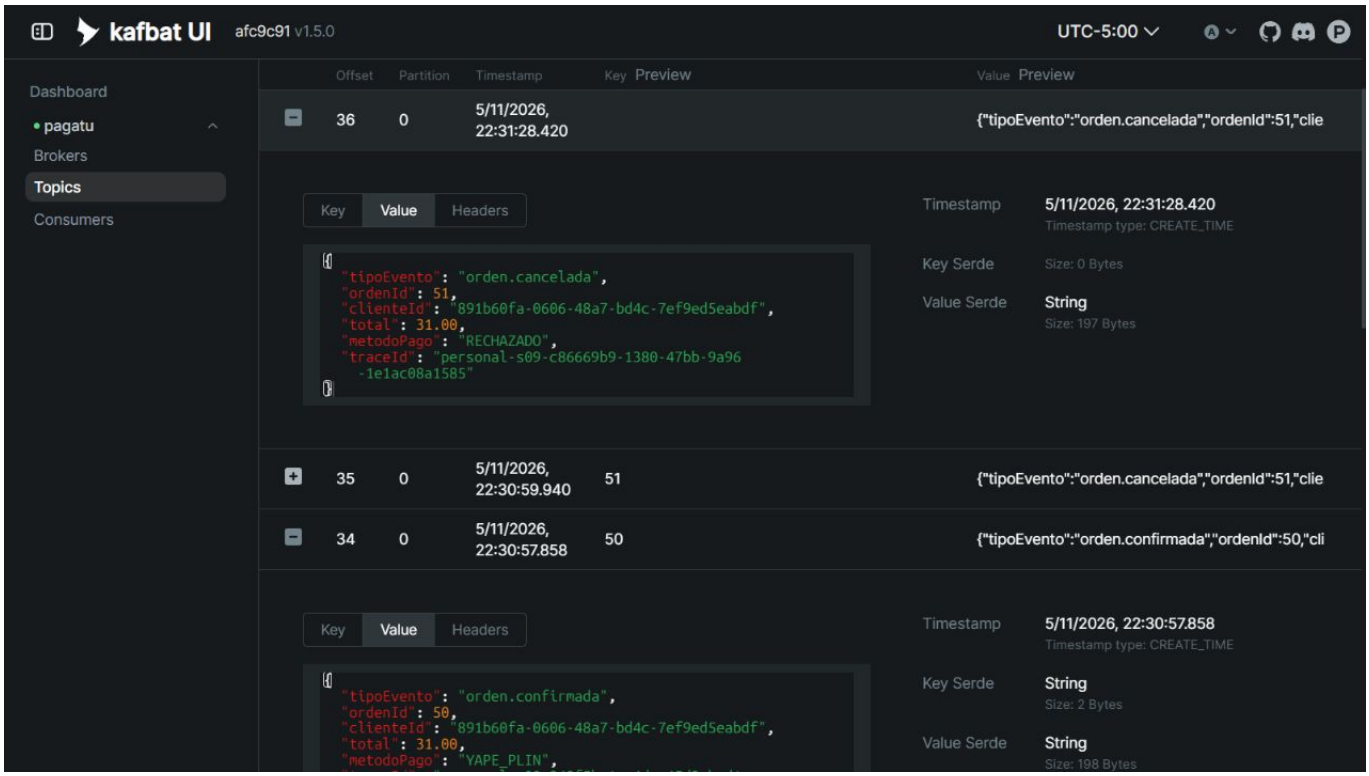
Duplicado de orden no duplica pago | {"orden": 56, "filas": 1}

Estado final de orden 56 | {"estado": "CANCELADA", "total": 31}

Compensacion duplicada no duplica aviso | {"orden": 56, "filas": 1}

Conflicto SQL real reproducido de forma aislada | {"error": "ERROR: duplicate key value violates unique constraint \"pagos_orden_id_key\"\nDETAIL: Key (orden_id)=(55) already exists.\n"}

Captura real



Captura íntegra, sin edición de resultados ni fechas. Kafka UI: orden.cancelada de la orden 51 repetida en offsets 35 y 36; la ejecución correspondiente se conserva en s09-primera-aprobada.json. La validación final puede usar otros IDs.. La salida original completa está vinculada en MkDocs.

Hallazgo y reflexión técnica

Error o hallazgo real

El INSERT repetido provoca una excepción UNIQUE real. El consumidor usa una operación atómica ON CONFLICT y revisa el número de filas afectadas; una comprobación solo en memoria no cubriría reinicios ni carreras.

Reflexión técnica

La confirmación de un efecto y la del mensaje pueden fallar en momentos distintos.

El productor puede no saber si el consumidor terminó y volver a enviar.

Aceptar al menos una vez exige que repetir el evento no repita el efecto.

La restricción UNIQUE hace persistente esa garantía en el pago.

La guardia de estado protege la compensación y evita duplicar el aviso.

El caso Knight Capital ilustra el daño que puede causar repetir acciones no controladas.

La garantía útil se verifica en el efecto de negocio, no contando entregas del broker.

Límite de alcance: esta entrega personal usa Pagatu; la adaptación al Proyecto Sello grupal CASAS se evalúa por separado. No se atribuye una defensa oral realizada ni una nota de evaluación.

Anexo · Feedback de la sesión

Campos personales pendientes de completar por Jannys. No se inventan respuestas sobre la experiencia, comprensión, esfuerzo o satisfacción en clase.

1. ¿Cuál es el aprendizaje más importante que te llevas de la clase de hoy?

2. ¿Qué punto de la clase te resultó más confuso o te dejó con dudas?

3. ¿Tienes alguna pregunta para la siguiente clase?

4. Nivel de comprensión: Entendido / Más o menos / Necesito ayuda.

5. ¿Cómo puede ayudarte el docente a comprender mejor el tema?

6. Participación: Muy comprometido/a / Comprometido/a / Poco comprometido/a.

7. Satisfacción con la clase: 1 a 10.